

Classification

→ Suppose I want to know if a newborn comes to a hospital, will it have seizures.

What do we have?

(Only for examples)

n training data points.

$i \rightarrow$ index.

$x^i \rightarrow$ Feature vectors

$$x^i = (x_1, x_2, \dots, x_d)^T \in \mathbb{R}^d$$

$y^i \rightarrow$ Label

$y^i \in \{-1, +1\}$ $+1 \rightarrow$ Yes seizure
 $-1 \rightarrow$ No seizure

$x_1 \rightarrow$ How much oxygen the newborn is breathing.

$x_2 \rightarrow$ Heartbeat of the newborn.

$x_3 \rightarrow$ Rate of movement of the newborn.

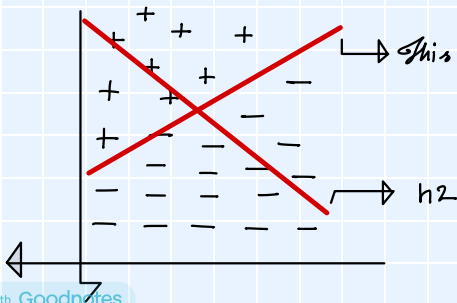
Training Data

$$D_n = \{(x^1, y^1), (x^2, y^2), \dots, (x^n, y^n)\}$$

$$x^i = (x_1^i, x_2^i, x_3^i)$$

Hypothesis:
Something that
has 1 on one
side & -1 on
another.

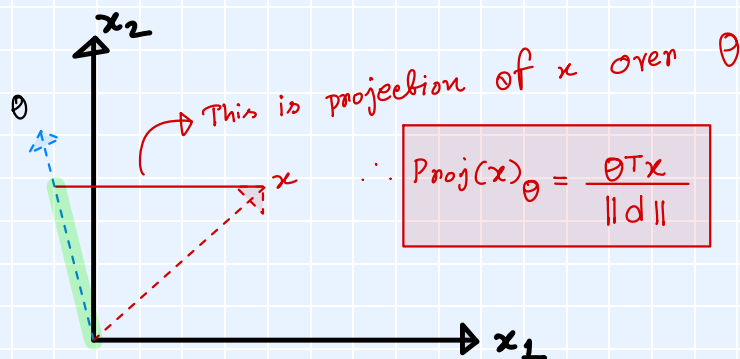
Goal: We need to find appropriate classifiers!



→ This is called a hypothesis (h_1)

Hence, h_1 is clearly better than h_2

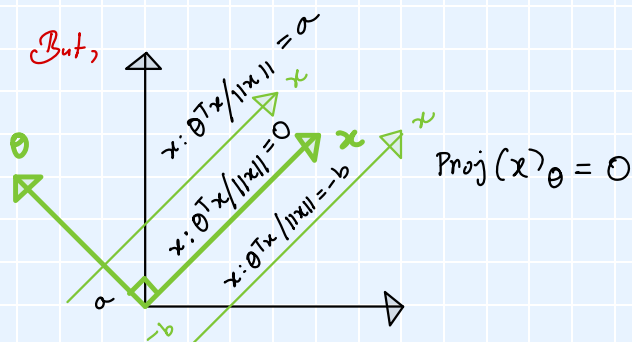
*Math Facts



$$\theta^T x \in \mathbb{R}^1$$

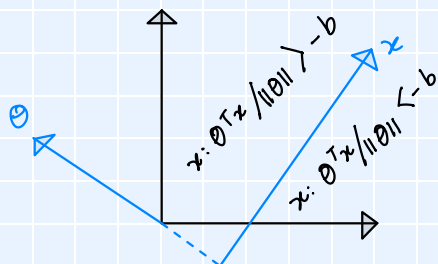
1x1 1x1

line = Projection being equal to a particular value.



$x: \theta^T x / \|x\| = 0$ means, I'm looking for x s of whose projection over θ is 0 & this describe a line.

This is another way of representing lines & this representation can help us form our hypotheses.



hypothesis: If we go near x , we'll have a label & if we go to the opposite direction, we'll have another label.

$$x: \theta^T x / \|\theta\| = -b$$

$$\text{or, } x: \theta^T x + b \|\theta\| = 0$$

$$x: \theta^T x + \theta_0 = 0$$

$h(x) = \text{sign}(\theta^T x + \theta_0) \rightarrow$ This is a sign classifier.

$$\Rightarrow " = \begin{cases} +1 & \text{if } \theta^T x + \theta_0 > 0 \\ -1 & \text{if } \theta^T x + \theta_0 \leq 0 \end{cases}$$

$$\Rightarrow h(\underbrace{x; \theta, \theta_0}_{\text{Parameters}}) = "$$

* How good is a classifier ?

$\rightarrow$ That gives the least amount of loss $\underbrace{L(g, a)}$
 $g \rightarrow$ guess
 $a \rightarrow$ actual value
 Opposite of utility

$$L = \begin{cases} 0 & \text{if } g = a \\ 1 & \text{else} \end{cases} \quad \text{This is symmetric loss.}$$

* Test Error (n' new points)

$$\mathcal{E}(h) = \frac{1}{n'} \sum_{i=1+n}^{n+n'} L(h(x^i), y^{(i)})$$

* Training Error ($\mathcal{E}_n(h) = \underbrace{\frac{1}{n} \sum_{i=1}^n L(h(x^i), y^i)}_{\text{To average out}})$

Perceptron

→ Historically it's important because it laid out the grounds for 'Neural-Network'

A linear classifier: $f: \mathbb{R}^d \rightarrow \{-1, 1\}$

$$h(x; \theta, \theta_0) = \text{sign}(\theta^T x + \theta_0) = \begin{cases} +1 & \text{if } h() > 0 \\ -1 & \text{if } h() \leq 0 \end{cases}$$

θ is always perpendicular to the hyperplane.

* This is also called a hypothesis * (h)

* A linear classifier separates data using a linear decision boundary

Hypothesis class $\mathcal{H}$ of all linear classifiers.

* 0-1 loss function

$$L(g, a) = \begin{cases} 0 & \text{if } g = a \\ 1 & \text{else} \end{cases}$$

* Training Error

$$E_n = \frac{1}{n} \sum_{i=1}^n L(h(x^{(i)}), y^{(i)})$$

* Perceptron is an algorithm that takes in data & throws out hypotheses.

Initialise $\theta = (0, 0, 0, \dots, 0)^T \in \mathbb{R}$
 $\theta_0 = 0$

$$\mathcal{D} = \{(x^i, y^i)\}_{i=1}^n$$

Goal = Learn a function that generalises beyond training data.

$w^T x + b \rightarrow$ A line in 2D
A plane in 3D
A hyperplane in higher dimension.

$y^{(i)}(w^T x^i + b) > 0 \rightarrow$ Linear separability

$$g(x) = w^T x + b$$

$$\hat{y} = \text{sign}(g(x))$$

*** Perceptron only works if data is linearly separable ***

- ① Correct classification means the point lies on the correct side of the hyperplane.
- ② The margin (signed distance) is positive.

The perceptron model

- ① A linear classifier.
- ② With a very specific learning rule.
- ③ Based on mistake-driven updates.

$h(x) = \text{sign}(w^T x)$ { Bias can be absorbed by augmenting x with a constant 1 }

* Perceptron loss: $\ell(w; x, y) = \max(0, -y w^T x)$

*** Perceptron Convergence Theorem:** If data is linearly separable, the perceptron algorithm converges in finite steps.

*** Margin & Convergence Speed**

$$R = \max_i \|R^i\|$$

γ = margin of the optimal separating hyperplane.

Number of mistakes: $\left(\frac{R}{\gamma}\right)^2$

*** NO LEARNING RATE ***

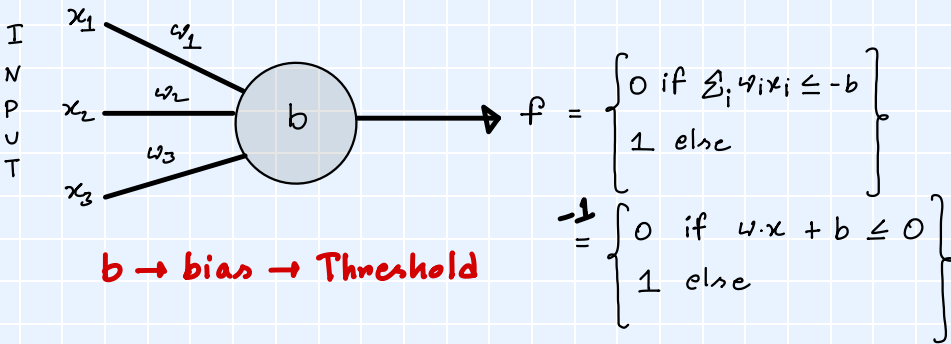
* Worked Example *

i	$x = (x_1, x_2)$	y
1	(2, 1)	+1
2	(1, -1)	-1
3	(2, -1)	-1

here,

$$X = \begin{pmatrix} 2 & 1 \\ 1 & -1 \\ 2 & -1 \end{pmatrix} \quad y = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

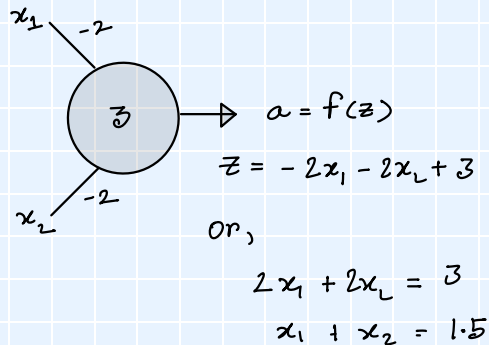
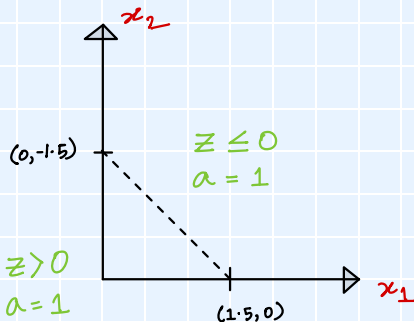
$$w^0 = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \rightarrow \text{initialised}$$



Activation function is the step function.

In a 2 variable perceptron

$a \rightarrow \text{activation}$



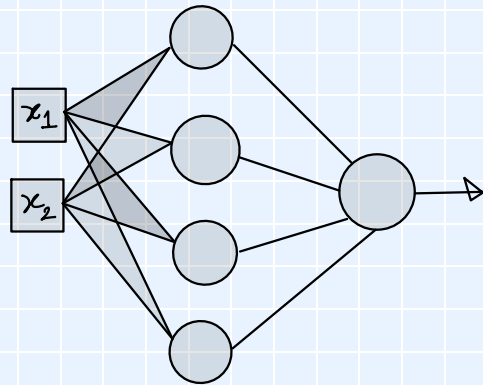
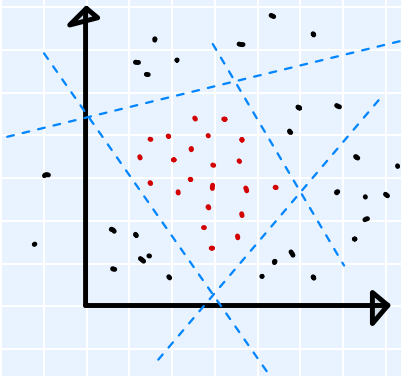
If inputs are binary (0,1) in this case

x_1	x_2	a
0	0	1
0	1	1
1	0	1
1	1	0

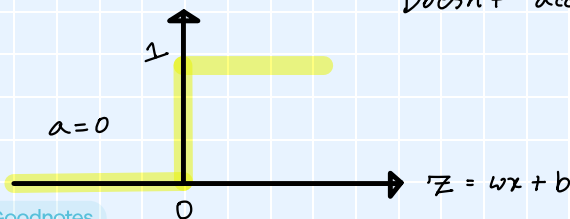
This is also similar to a NAND gate.

which is an universal gate & that means perceptrons are universal.

A single layer of perceptron is a linear classifier & can separate linearly separable data but if our data is not linearly separable, we can use multiple perceptrons to classify those.



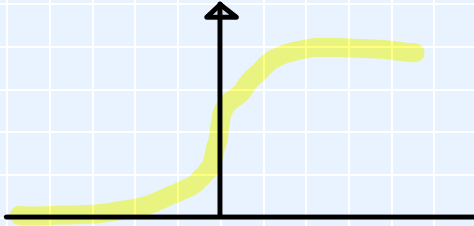
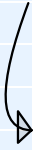
Activation Function



Doesn't account for small change

But we want to account for small changes in weights
& that's how we end up using the sigmoid function.

$$\sigma = \frac{1}{1 + e^{-z}}$$



for j in range $(0, T^c)$:

changed = False;

for j in range $(0 \text{ to } n)$:

if $(y^{(i)} (\theta^T x^{(i)} + \theta_0) \leq 0$:

$\theta = \theta + y^{(i)} x^{(i)}$;

$\theta_0 = \theta_0 + y^{(i)}$;

changed = True;

if not changed:

break;

return θ, θ_0

$n \rightarrow$ # Datapoints

$T^c \rightarrow$ Till some point
I don't want to
go to.

$$y^{(i)} = y_i$$

What does an update do?

$$y_i ((\theta + y_i x_i)^T x_i + (\theta_0 + y_i))$$

$$= y_i (\theta^T x_i + \theta_0) + (y_i)^2 (x_i^T x_i + 1)$$

$$= \underbrace{y_i (\theta^T x_i + \theta_0)}_{\text{Our initial equation}} + \underbrace{(y_i)^2 (x_i^T x_i + 1)}_{\text{Strictly positive}}$$

Our initial
equation

Strictly
positive

* Breaking Down a bit more

$$x = (x_1, x_2, \dots, x_n)$$

$$w = (w_1, w_2, w_3, \dots, w_n)$$

$b = \text{bias}$

$$z = w \cdot x + b$$

Actual targets,

$$y = (y_1, y_2, \dots, y_n)$$

Predicted Targets

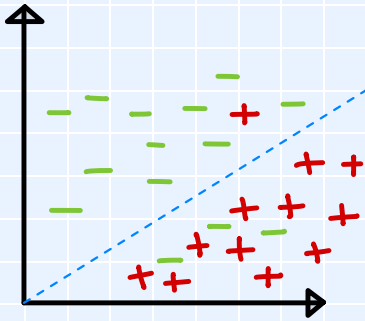
$$\hat{y} = (\hat{y}_1, \hat{y}_2, \hat{y}_3, \dots, \hat{y}_n)$$

$$= \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

* All of these basically mean that perceptron draws a line (2D) or hyperplane (nD).

* Everything on one side is class 1, the other side is class '0'.

* Classifier Quality



→ This dataset is not linearly separable; so, a single line would not be able to separate it.

* Numerical Example

$$\bar{x} = \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \quad \bar{y} = \begin{pmatrix} +1 \\ -1 \end{pmatrix}$$

Initialisation,

$$\bar{w} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \quad \hat{\bar{y}} = \text{sign}(\bar{w}^T \bar{x}) \quad \text{correctness: } \bar{y}(\bar{w}^T \bar{x}) \leq 0?$$

Now, calculation in a per feature-set basis,

$$\hat{y}^1 = w^T x = (0 \ 0) \begin{pmatrix} 2 \\ 1 \end{pmatrix} = 0 \ (-1)$$

Correctness,

$$y \hat{y} = +1 \times 0 = 0 \leq \text{Mistake}$$

$$\therefore w^1 = w^0 + yx = \begin{pmatrix} 0 \\ 0 \end{pmatrix} + (+1) \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

Now for point 2,

$$\begin{aligned} \hat{y}^{(2)} &= w^1 T x = (2 \ 1) \begin{pmatrix} 1 \\ -2 \end{pmatrix} \\ &= 2 + -2 = 0 \end{aligned}$$

$$y^2 \hat{y}^2 = (-1)(0) = 0 \leq 0 \rightarrow \text{Misclassified}$$

$$\begin{aligned} \therefore w^2 &= w^1 + yx = \begin{pmatrix} 2 \\ 1 \end{pmatrix} + (-1) \begin{pmatrix} 1 \\ -2 \end{pmatrix} \\ &= \begin{pmatrix} 2 \\ 1 \end{pmatrix} + \begin{pmatrix} -1 \\ +2 \end{pmatrix} \\ &= \begin{pmatrix} 1 \\ 3 \end{pmatrix} \end{aligned}$$

Checking all points with our new weights,

$$w^{(2)T}x = (1 \ 3) \begin{pmatrix} 2 \\ 1 \end{pmatrix} = 2 + 3 = 5$$

$$y_1(w^T x^1) = 5 > 0 \quad \checkmark \text{ verified}$$

Iteration stops!